AI Engineer Transition — Phase 2 Study Notes

LLM Fundamentals: tokens, sampling, local models, prompting, embeddings. Lessons 2.1–2.7, all completed and verified hands-on on local hardware (RTX 2050, Ollama + Qwen3).

2.1 — What an LLM is: tokens, next-token prediction

DONE

Key ideas

- An LLM is a function: given tokens so far, predict a probability distribution over the next token. Everything else (chat, reasoning, code) is this one operation applied repeatedly.
- Tokens are sub-word chunks from a fixed vocabulary — not whole words. Built via BPE: start from characters, repeatedly merge the most frequent adjacent pair, stop at a target vocabulary size (e.g. exactly 50,000 — chosen upfront, not an estimate).
- ~0.75 words per token is a measured average for English, not a designed constant — other languages/scripts can have very different ratios.
- Generation is iterative: predict next token → append it → feed the WHOLE sequence back in → repeat. Nothing is planned ahead; each token is a local decision.
- Chain-of-thought works because each written intermediate step becomes part of the context the model conditions on for the next token — the writing IS the computation, not hidden reasoning.

Java/Spring analogy

Java / Spring	Python / AI
`String generate(List<Token> ctx) in a loop`	`next token prediction, repeated`
`enum / fixed lookup table`	`fixed vocabulary (e.g. 50,000 entries)`

Takeaway: Verified with the OpenAI tokenizer: common words = 1 token, rare/long words split into several (e.g. 'unbelievable' -> ['un','believ','able']).

2.2 — Context window, temperature, sampling

DONE

Key ideas

- Context window = max tokens (input+output combined) per call. The model is stateless — every call resends the FULL conversation; there's no server-side memory between calls.
- Raw output per step: one logit (raw score) per vocabulary entry (~50,000 numbers) → softmax converts to probabilities summing to 1.0 → sampling picks one token from that distribution.
- Temperature reshapes logits BEFORE softmax: low = sharper/more deterministic (near 0 = always pick the top token, i.e. greedy decoding); high = flatter/more random, more risk of nonsense.
- top_p (nucleus sampling): after temperature + softmax, keep only the smallest top-ranked set of tokens whose probabilities sum to p; sample only from that shortlist. Excludes the implausible tail regardless of temperature.
- Order of operations: logits → temperature scaling → softmax → top_p filtering → renormalize → sample.

Core mechanism

```
logits -> /temperature -> softmax -> probabilities (sum to 1.0)
probabilities -> sort desc -> keep until cumulative >= top_p -> renormalize -> sa
mple
```

Takeaway: For deterministic/structured tasks (e.g. JSON field extraction): temperature near 0 — there's exactly one correct answer, no room for creative variation. Higher temperature suits tasks with genuinely many valid continuations (creative writing).

2.3 — Install Ollama, run Qwen locally, verify GPU use DONE

Key ideas

- Ollama runs as an always-on background service/daemon, serving an HTTP API at localhost:11434. `ollama run` is just one interactive client of that same server — your own Python code is another client of the identical server.
- Models load on-demand on first request naming them, stay loaded briefly (idle timeout, shown as the UNTIL column in `ollama ps`), then unload automatically to free memory.
- A model is made of many stacked layers. Ollama loads as many layers as fit in VRAM; the rest run on CPU — graceful degradation across two separate memory pools (VRAM + RAM), unlike Java's single-heap all-or-nothing OutOfMemoryError.

Java/Spring analogy

Java / Spring	Python / AI
`Always-on app server (Tomcat)`	`Ollama background service on :11434`
`@Lazy bean, first-use init`	`model loaded into memory on first request`
`Single heap -> OutOfMemoryError`	`two pools (VRAM+RAM) -> graceful partial split`

Takeaway: Measured on this hardware (RTX 2050, 4GB VRAM): qwen3:1.7b ran 100% GPU (fully fits); qwen3:4b split ~34%/66% CPU/GPU (partially fits) -- confirmed via `ollama ps`, with the 1.7b model visibly faster in practice.

2.4 — Call Ollama from Python (with config switch) DONE

Key ideas

- Ollama exposes an OpenAI-compatible API, so the standard `openai` Python package works, just pointed at base_url="http://localhost:11434/v1".
- `.env` file holds config as plain KEY=VALUE text (external, not source code). `load_dotenv()` reads it and injects values into the process's real environment variables. `os.environ.get("KEY", default)` then reads from the environment generically -- same mechanism a real deployment would use, regardless of whether values came from .env or the OS.
- client = OpenAI(base_url=..., api_key=...) is a shared, singleton-style configured client, built once from externally-supplied values, then imported wherever needed.
- response.usage exposes prompt_tokens / completion_tokens / total_tokens. Qwen3 is a 'thinking' model -- completion_tokens includes hidden reasoning tokens not shown in the printed output, which matters for cost/latency later (P7).

Java/Spring analogy

Java / Spring	Python / AI
`application.yml + @Value injection`	`.env + os.environ.get(key, default)`
`System.getenv(key)`	`os.environ.get(key)`
`Dev-only env file loader plugin`	`python-dotenv's load_dotenv()`
`Singleton @Bean RestTemplate/WebClient`	`shared `client` object built once from config`

Example

```
load_dotenv()
MODEL = os.environ.get("LLM_MODEL", "qwen3:4b")
client = OpenAI(base_url=BASE_URL, api_key=API_KEY)

response = client.chat.completions.create(model=MODEL, messages=[...])
response.choices[0].message.content   # generated text
response.usage                        # token counts
```

Takeaway: Verified end-to-end: changing ONLY .env's LLM_MODEL (qwen3:4b -> qwen3:1.7b) and rerunning the CLI chat app with zero code changes correctly switched the serving model -- this is the entire point of externalizing config instead of hardcoding the model name.

2.5 — Prompting: roles, examples, structure

DONE

Key ideas

- Every chat call is a list of role-tagged messages: system (persistent behavior/constraints, set once), user (human input), assistant (model's past replies) -- not a single string.
- The model is stateless (2.2): conversation 'memory' is really just resending the entire messages list every call.
- Few-shot prompting: including example user/assistant pairs BEFORE the real question, demonstrating the exact input->output pattern wanted. Often more reliable than describing the format in words, since the model pattern-matches concrete examples rather than interpreting an abstract instruction.
- IMPORTANT caveat (observed directly): fabricated assistant turns work because the model has NO way to distinguish a genuinely-generated past turn from one you typed yourself -- it just conditions on whatever tokens are present, with zero concept of provenance. But few-shot also skews the CONTENT distribution, not just the format: in a live test, a system-prompt-only call correctly answered MIXED for a mixed-sentiment sentence, but the few-shot version (whose two examples were only POSITIVE/NEGATIVE, never MIXED) answered POSITIVE instead -- the model was statistically nudged toward only the categories it was shown, even though MIXED was still a technically allowed answer.

Java/Spring analogy

Java / Spring	Python / AI
`@ControllerAdvice (shapes every response)`	`system message (set once, applies throughout)`
`Example-based unit test fixtures`	`few-shot user/assistant example pairs`

Takeaway: Few-shot prompting is powerful for format reliability, but silently also restricts outputs toward whatever categories/patterns your examples happened to demonstrate -- include a representative example of EVERY valid output category, not just the common ones.

2.6 — Embeddings: concept + generate them locally

DONE

Key ideas

- An embedding turns text into a fixed-length vector (e.g. 768 floats) such that texts with similar MEANING end up numerically close -- regardless of shared vocabulary. Different from tokenization (2.1), which is about chopping text into vocabulary pieces for processing.
- Cosine similarity (-1 to 1) measures closeness by the angle between two vectors, ignoring magnitude: 1 = same direction/meaning, 0 = unrelated, -1 = opposite.
- This is the foundation of semantic search (Phase 4 / RAG): embed a query and every document, then rank by cosine similarity -- find what MEANS similar, not what shares keywords.
- nomic-embed-text and qwen3 are structurally different model types -- a generative model predicts next tokens (chat.completions endpoint); an embedding model outputs one fixed vector per input (embeddings endpoint). Neither can substitute for the other's job.

Core mechanism

```
cosine_similarity(a, b) = dot(a, b) / (norm(a) * norm(b))
```

Java/Spring analogy

Java / Spring	Python / AI
`hashCode() for bucketing/equality`	`embedding vector for SEMANTIC distance`
`Two separate interfaces: TextGenerator vs TextEncoder`	`chat.completions vs embeddings endpoint`

Takeaway: Measured locally with nomic-embed-text (768-dim): 'cat sat on mat' vs 'feline rested on rug' scored 0.702 similarity despite near-zero shared words; stock-market vs equity-prices scored 0.664; every cross-topic pairing (e.g. either vs a pizza sentence) sat around 0.26-0.38 -- clear, measured proof that embeddings capture meaning, not word overlap.

2.7 — PROJECT -- CLI chat (local model, switchable) DONE

Key ideas

- Built a real interactive CLI chat app maintaining its own running `messages` list across turns -- proving in practice that conversation memory is entirely client-maintained (2.2, 2.5), not provided by the server.
- Implemented /exit, /reset (clears back to just the system prompt), /history (prints the full message list), and a sliding-window guard (drop oldest user/assistant pair once the list exceeds ~20 messages) -- a simplified stand-in for real context-window management (full token counting comes in 3.6).
- Verified multi-turn memory worked for real (a multi-turn Sinhala-language exchange correctly referenced earlier turns), verified /reset genuinely wipes context (post-reset question got a context-free generic answer), and verified the .env model-switch mechanism from 2.4 works end-to-end inside a full application, not just a one-off script.
- Caught and corrected an anti-pattern during review: switching models by editing the DEFAULT VALUE inside llm_config.py (code change) instead of editing .env (config change) defeats the whole purpose of externalized config -- re-verified the correct way afterward.

Takeaway: Phase 2 capstone complete: a working stateful LLM application with real conversation history, command handling, context-window guarding, and a verified zero-code-change provider switch.

What Phase 2 built, end to end

- Fluency in how LLMs actually generate text: tokens, sampling, temperature, top_p.
- A fully working local inference stack: Ollama + Qwen3, with verified GPU utilization.
- A reusable, externally-configured LLM client (swap providers/models with zero code change).
- Practical prompt design: roles, system prompts, few-shot examples, and their hidden content-bias risk.
- Embeddings and cosine similarity as the basis for semantic search, ahead of Phase 4 (RAG).
- A real stateful CLI application tying all of the above together.

Next: Phase 3 -- Build with APIs (chat basics, streaming, structured JSON output, tool calling, agent loops), building directly on this config layer and messages structure.